CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 37

Dr. Asad Mahmood

April 29, 2026

Lecture Outline

- Announcements (Assignment and Quiz 5)
- Outline of Previous Lecture:
  - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
    - 5.2 Memory Technologies
    - 5.3 The Basics of Caches

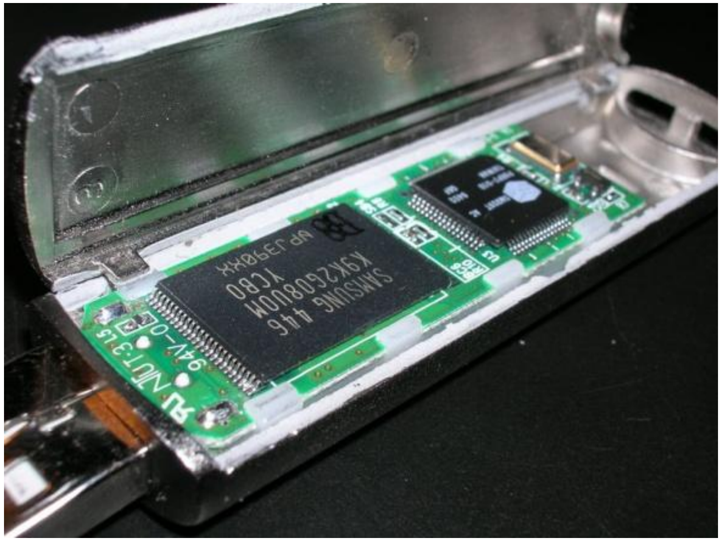
- Outline of Today's Lecture:
  - **Chapter 5 – Large and Fast: Exploiting Memory Hierarchy**
    - 5.3 The Basics of Caches

Chapter 5

Large and Fast: Exploiting Memory Hierarchy

Flash Storage

- Nonvolatile semiconductor storage
  - 100× – 1000× faster than (hard) disk
  - Smaller, lower power, more robust
  - But more $/GB

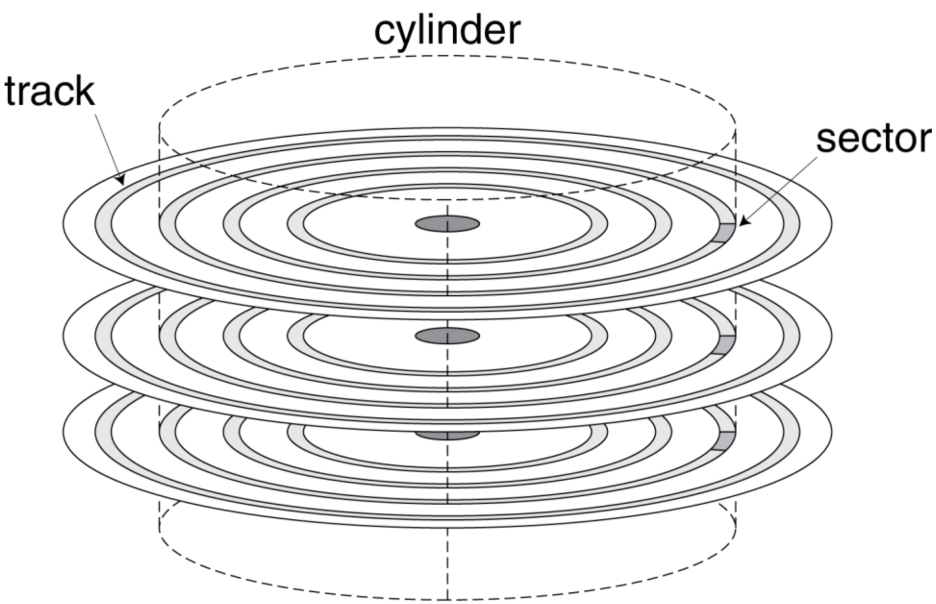


Flash Types

- NOR flash: bit cell like a NOR gate
  - Random read/write access
  - Used for instruction memory in embedded systems
- NAND flash: bit cell like a NAND gate
  - Denser (bits/area), but block-at-a-time access
  - Cheaper per GB
  - Used for USB keys, media storage, …
- Flash bits wears out after 1000's of accesses
  - Not suitable for direct RAM or disk replacement
  - Wear leveling: remap data to less used blocks

(Hard) Disk Storage

- Nonvolatile, rotating magnetic storage



Disk Sectors and Access

- Each sector records
  - Sector ID
  - Data (512 bytes, 4096 bytes proposed)
  - Error correcting code (ECC)
    - Used to hide defects and recording errors
  - Synchronization fields and gaps
- Access to a sector involves
  - Queuing delay if other accesses are pending
  - Seek: move the heads
  - Rotational latency
  - Data transfer
  - Controller overhead

Disk Access Example

- Given
  - 512B sector, 15,000 rpm, 4ms average seek time, 100MB/s transfer rate, 0.2ms controller overhead, idle disk
- Average read time
  - 4ms seek time
    + ½ / (15,000/60) = 2ms rotational latency
    + 512 / 100MB/s = 0.005ms transfer time
    + 0.2ms controller delay
    = 6.2ms
- If actual average seek time is 1ms
  - Average read time = 3.2ms

Disk Comparison

- Two primary differences between magnetic disks and semiconductor memory technologies (RAM, Flash)
- Magnetic disks are nonvolatile like flash, but unlike flash there is no write wear-out problem.
- However, flash is much more rugged and hence a better match for personal mobile devices.

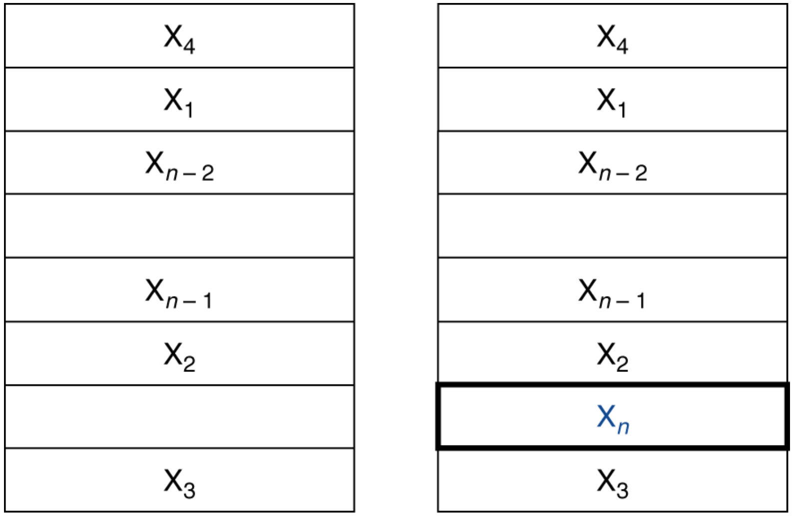
Chapter Outline

Chapter 5 – Large and Fast: Exploiting Memory Hierarchy

- 5.1 Introduction
- 5.2 Memory Technologies
- 5.3 The Basics of Caches
- 5.4 Measuring and Improving Cache Performance
- 5.5 Dependable Memory Hierarchy
- 5.6 Virtual Machines
- 5.7 Virtual Memory
- 5.8 A Common Framework for Memory Hierarchy
- 5.6 onwards (self-reading)

Cache Memory

- Cache memory
  - The level of the memory hierarchy closest to the CPU
- Assume if the cache contains recent data accesses X_1, …, X_{n-1}, and the processor requests X_n

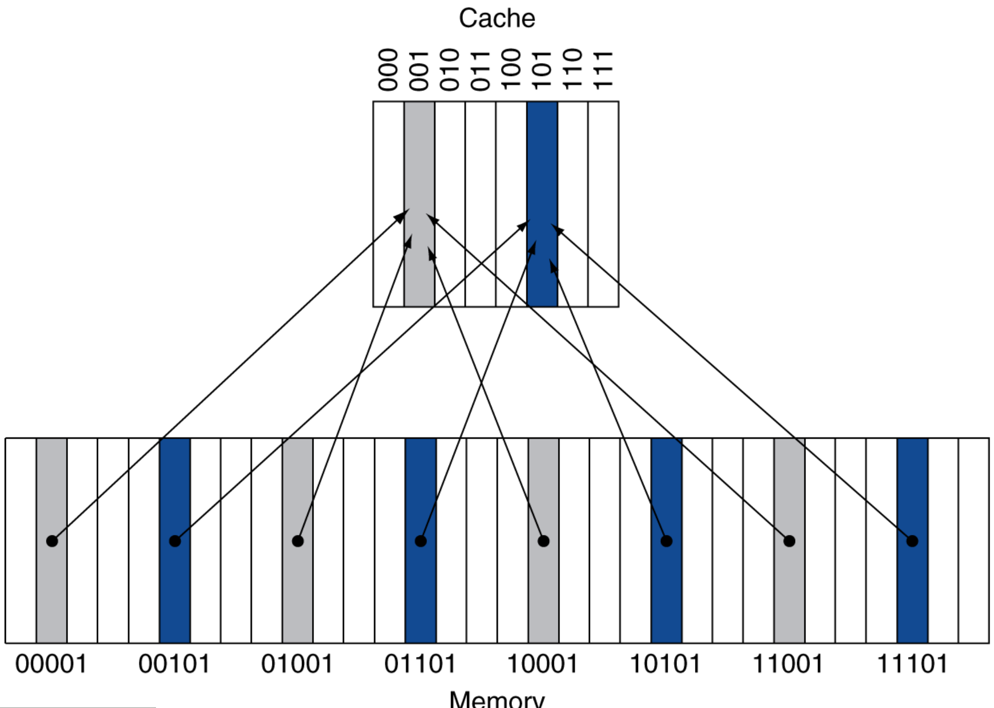


a. Before the reference to X_n b. After the reference to X_n

- How do we know if the data is present?
- Where do we look?

Direct Mapped Cache

- A good strategy would be to fix the cache location of data based on its memory address
- Direct mapped: only one choice
  - (Block address) modulo (#Blocks in cache)



- If #Blocks is a power of 2, one can use low-order log2(#Blocks) address bits

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store the high-order bits from block-address as well
  - Called the tag
- What if there is no valid data in a location?
  - Valid bit: 1 = present, 0 = not present
  - Initially 0

Cache Example

- Consider an 8-blocks, 1 word/block, direct mapped cache
- Below is a sequence of nine memory references to the initially empty cache

Decimal address of reference	Binary address of reference
22	10110_{two}
26	11010_{two}
22	10110_{two}
26	11010_{two}
16	10000_{two}
3	00011_{two}
16	10000_{two}
18	10010_{two}
16	10000_{two}

Cache Example

- Initial state

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	**Y**	**10**	**Mem[10110]**
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Index	V	Tag	Data
000	N		
001	N		
010	**Y**	**11**	**Mem[11010]**
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Index	V	Tag	Data
000	N		
001	N		
010	Y	11	Mem[11010]
011	N		
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

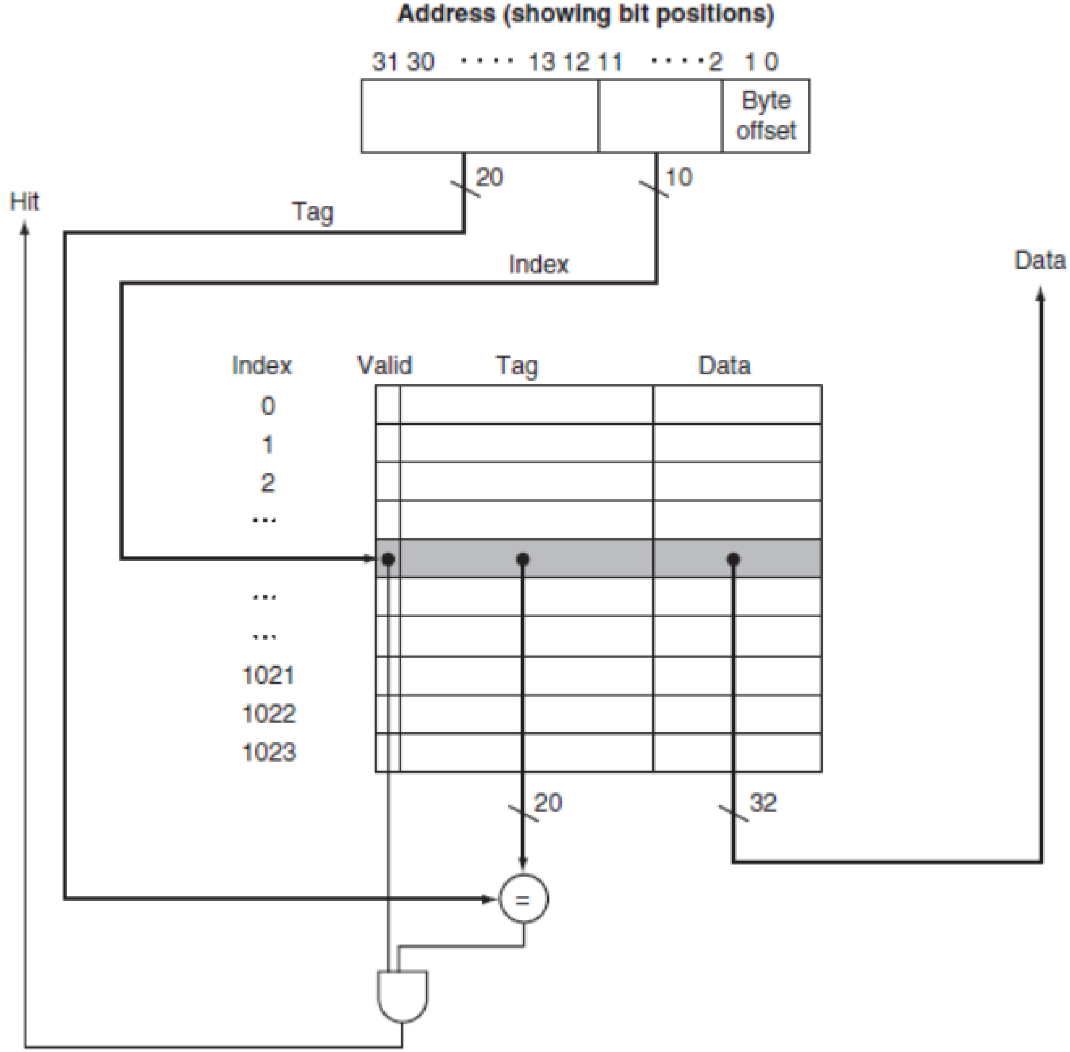
Index	V	Tag	Data
000	**Y**	**10**	**Mem[10000]**
001	N		
010	Y	11	Mem[11010]
011	**Y**	**00**	**Mem[00011]**
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Cache Example

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	**Y**	**10**	**Mem[10010]**
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Address Subdivision



Example: Total bits in Cache

- Consider:
  - 32-bit address size
  - Direct-mapped cache
  - Cache size = 2^n blocks
  - Block size = m words
  - Valid field size = 1 bit
- The naming convention is to exclude the size of the tag and valid field and to count only the size of the data.
- Example: How many total bits are required for a direct-mapped cache with 16 KiB of data and four-word blocks, assuming a 64-bit address?

Example: Mapping Address to Multiword Cache

- Example: Mapping Address to Multiword Cache Block
- 64 blocks, 16 bytes/block
  - To what block number does address 1200 map?